

OSTIM TECHNICAL UNIVERSITY

Department of Software Engineering

SENG 456

AGENT ORCHESTRATION AND MULTIMODAL SYSTEMS

INDIVIDUAL TERM PROJECT

SUBMISSION FORM

PROJECT TITLE

PURPLE-LOOP — A Dynamic Multi-Agent Orchestration Architecture for Adversarial Security Evaluation (Red vs Blue vs Audit)

Full Name	Mohammed Mustafa Kareem
Student ID	230208878
Department / Program	Software Engineering
Email Address	230208878@ostimteknik.edu.tr
GitHub / Project URL	myskillos.com/project/490fe3ca-64b6-4831-b7e5-17a67adf7724
Demo / Video URL (if available)	Not provided
Submission Date	19 August 2026

PROJECT SUBMISSION URL

myskillos.com/proje

Upload this form and the project files to the specified address.

Brief Description

PURPLE-LOOP is a dynamic multi-agent orchestration in which an attacker agent, a defender agent and an independent auditor argue over a shared state object until the auditor certifies convergence or the round budget is exhausted. Control flow is decided at runtime by two LLM judgments — the auditor’s verdict and the router’s routing decision. The deterministic code only *validates* moves; it never chooses them.

Contents

Brief Description	1
1 Project Summary	2
2 Input, Output, and Usage	2
3 System Design	3
4 Agents, Tools, and Integrations	5
5 Implementation and Execution	6
6 Tests and Results	6
7 Security and Responsible Use	7
8 Conclusion and Future Work	8
9 References and Submission Checklist	9
10 Student Declaration	10

1 Project Summary

Provide a general overview of the project in a few short paragraphs.

The problem the project aims to solve

A classic security review is static: it walks a checklist, matches patterns with `if/else`, and emits a score. Such a pipeline cannot reason about novel attack composition, cannot argue with itself, and cannot decide that a particular question deserves a second look. Its output is also unfalsifiable — a defence is recorded as effective because it was proposed, never because it was tested.

PURPLE-LOOP replaces the checklist with a structured disagreement. The core problem it solves is making the *control flow itself* a reasoning product: which agent speaks next, and what it is asked to do, is derived from the current epistemic state of the engagement rather than from a fixed script.

The purpose of the project and its target users

The purpose is to evaluate a described system architecture through adversarial debate and to produce a security posture assessment whose every claim is traceable to a recorded verdict. Target users are software engineering and security students running architecture reviews as a tabletop exercise, and engineers who want a structured second opinion on a design document before implementation. It is a teaching and design-review instrument, not a penetration-testing tool.

Brief project summary: How does the system work overall, and what does it produce?

The user supplies a free-text description of a (fictional) system. A **Router** (chief) agent inspects a compact digest of shared state and selects the next agent from a formally computed set of legal moves, writing that agent a specific mandate. **Recon** decomposes the target into assets and trust boundaries. **Red Team** produces attack hypotheses in two lanes — software/logic and social engineering — which pass through a **Safety Guardian** filter before entering state. **Blue Team** answers with layered countermeasures. The **Security Auditor** issues one of six verdicts per finding/mitigation pair; a **CHALLENGE** verdict sends the defender's own patch back to the attacker as the next target. The loop ends on convergence or budget exhaustion, after which the auditor authors the final report.

Output: a Markdown engagement report containing an agent-authored executive summary, a 0–100 security score with weighted breakdown, the full routing history, the disagreement history, a per-agent calibration ledger, the safety-guardian log, and a complete `final_state.json` that allows any turn to be replayed.

Application domain	Cybersecurity — threat modelling and architecture review (tabletop, fictional targets)
Languages / technologies used	Python 3.10+ (standard library only), Markdown/YAML agent specifications, JSON Schema, LaTeX
Models or services used	Claude (<code>claude-sonnet-5</code>) through the Anthropic Messages API in <code>-live</code> mode; a deterministic mock brain for offline, key-free execution
Current project status	Complete and runnable. 10/10 architectural self-tests pass; a full 12-round engagement executes end to end

2 Input, Output, and Usage

Summarize what the system receives, what it produces, and how it is used.

Primary inputs

(1) **target_spec** — a free-text description of a fictional system architecture: components, data flows, stated controls and known constraints. The bundled example is **sample_target.md**, a fictional fintech named “NovaPay”. (2) **max_rounds** — the turn budget (6–12 recommended). No agent may extend it.

Primary outputs

(1) **sample_run_report.md** — the engagement report: agent-authored executive summary, weighted score breakdown, routing history with the legal moves available at each turn, findings table, disagreement history, calibration ledger, guardian log, outstanding risk. (2) **final_state.json** — the complete **EngagementState** object, including the append-only patch history for replay. (3) A single composite security score on 0–100 with a letter grade.

Brief usage scenario: Main steps from the user request to the result

1. The user writes or selects an architecture description and sets a round budget.
2. The orchestrator initialises **EngagementState** and calls the Router.
3. The Router returns **next_agent** plus a mandate; the orchestrator verifies the choice lies inside the legal-move set and runs that agent against its state projection.
4. Attacker output is routed through the Safety Guardian; the resulting patch is validated against five invariants and applied or rejected with a logged reason.
5. The scorecard and convergence metrics are recomputed; the loop repeats from step 3.
6. On convergence or budget exhaustion the Auditor authors the final report, which is written to disk together with the final state.

A sample input and its corresponding output

Input (excerpt from sample_target.md): “Support desk — outsourced to a BPO partner. Agents verify callers using name, date of birth, and the last four digits of the registered phone number, then may trigger a password reset from the admin console.”

Output (excerpt from the engagement report): Finding F1 (lane: social) — “*Support desk identity verification relies on knowledge that is not secret.*” Likelihood 4 / Impact 4, confidence 0.67. Round 3: Blue proposes mitigation M1 with a preventive and a containment layer only. Round 4: the Auditor returns **INSUFFICIENT** — “*No detection layer: if prevention fails the event is invisible.*” Round 6: Blue submits M4 (version 2) with a detection layer added. Round 7: verdict **SOUND**. Final engagement score: **82.6/100, grade B**.

3 System Design

Describe the architecture only to the extent that it applies to your project.

Why did you choose this approach? Briefly justify the choice of an agent, workflow, or traditional software

A fixed workflow (Recon → Red → Blue → Audit → end) was rejected because it wastes turns on solved areas and starves contested ones, and because it can never revisit a defence once proposed. Traditional software was rejected because the judgments involved — “is this attack reachable given these trust boundaries?”, “does this patch actually close the finding?” — are not expressible as rules over a finite feature set.

The agent approach was chosen specifically so that *routing* could be a reasoning step. Two LLM decisions drive control flow: the Auditor’s verdict on each finding/mitigation pair, and the Router’s choice of the next agent and its mandate. Determinism is retained where it belongs: the legal-move set and the five state invariants are computed in code, so the LLM exercises judgment inside a formally bounded space rather than an unbounded one.

The system’s main components and their responsibilities

Six agents plus a deterministic orchestration harness. The Router (chief) selects the next agent and writes its mandate. Recon builds the asset and trust-boundary inventory. Red Team generates attack hypotheses in two lanes. Blue Team produces three-layer countermeasures. The Security Auditor scores, contests, routes and reports. The Safety Guardian filters every attacker output. The harness owns state: it validates patches against the invariants, computes the legal-move set, recomputes the scorecard, and grades each routing prediction. Red and Blue never communicate directly — all inter-agent communication is mediated by shared state, which makes every turn inspectable and replayable.

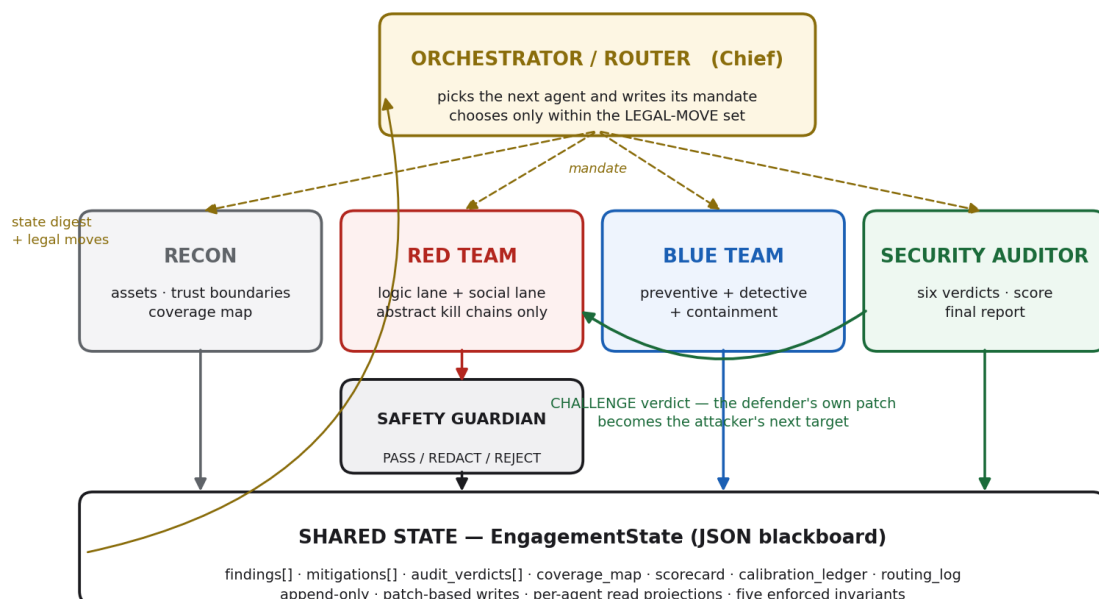
The main data or task flow between components

A single JSON object, **EngagementState**, is the only channel. Each agent receives a *projection* of it (only entitled fields, with prior-round findings compacted to a headline) and returns a *patch*, never a full rewrite. Patches are applied by the orchestrator under five invariants: findings are append-only; every mitigation must reference a live finding; the auditor may not author findings or mitigations; only the guardian may write attacker content; no agent may extend the round budget. Every applied or rejected patch is recorded in **patch_history**.

Architecture diagram:

PURPLE-LOOP — orchestration map

Control flow is decided per turn by two LLM judgments: the Auditor’s verdict and the Router’s routing decision.
agents never hand off to each other — every turn returns to the Router



4 Agents, Tools, and Integrations

If your project does not use agents or external tools, you may write “not used” in the relevant rows.

Component / agent / tool	Brief responsibility	Information received or produced
orchestrator-router (Chief)	Chooses the next agent and writes its mandate; may choose only from the legal-move set	Receives a compact state digest and the legal moves; produces next_agent , mandate , lane , rationale and a falsifiable prediction
recon-analyst	Decomposes the target; surfaces the human and process assets the specification omitted	Receives target_spec ; produces asset_inventory , trust_boundaries , coverage_map , unknowns
red-team	Attacker. Generates abstract attack hypotheses in the logic and social lanes; attacks deployed patches on a CHALLENGE mandate	Receives assets, boundaries, existing mitigations and verdicts; produces findings[] with ATT&CK IDs, likelihood, impact, confidence, novelty
blue-team	Defender. Produces preventive, detective and containment layers per finding and declares the assumptions each control rests on	Receives findings and verdicts; produces mitigations[] with assumptions , does_not_stop , residual-risk claim
security-auditor	Independent judge; scores, contests, instructs, decides convergence, authors the report	Receives the full state except target_spec ; produces audit_verdicts[] , scorecard , calibration_ledger , final_report
safety-guardian	Filters every attacker output before the state write: PASS, REDACT or REJECT	Receives candidate findings; produces sanitized findings and guardian_log

APIs, data sources, MCP, Skills, models, or other integrations used (if any)

Anthropic Messages API (**claude-sonnet-5**) in **-live** mode; no key is required for the default mock mode. The agent team is packaged as a Skill in the Myskillos / Claude layout (**CLAUDE.md** plus **.claude/agents/*.md**), generated from the prompt document by **python purple_loop.py -build-package** so the report and the uploaded skill cannot drift apart. Conceptual references only, with no runtime dependency: the MITRE ATT&CK technique vocabulary and the STRIDE threat categories. No external data sources, databases or network calls are used at run time.

Important decisions, permission boundaries, or operations requiring human approval (if any)

Three boundaries are enforced structurally rather than by convention. (1) The Safety Guardian sits between the attacker and shared state; attacker output that has not been cleared is rejected by the patch validator, so the filter cannot be bypassed even by a malformed agent response. (2) Separation of powers: the auditor is structurally prevented from authoring findings or mitigations, so the judge cannot manufacture the evidence it scores. (3) Human escalation: when no legal move would meaningfully advance the engagement — for example when the target description is too thin to support further analysis — the Router returns **next_agent: "HUMAN"** with one specific question and the engagement pauses rather than fabricating depth. The round budget is likewise a human-only parameter; no agent can extend it.

5 Implementation and Execution

Source code URL	Included in this submission (<code>implementation/</code>). Myskillos project: myskillos.com/project/490fe3ca... (agent team exported to <code>myskillos_export/</code>)
Main execution command	<code>python purple_loop.py -target sample_target.md</code>
Required core dependencies	Python 3.10+ with the standard library only. Optional: <code>anthropic</code> (needed only for <code>-live</code>)
Important files or folders	<code>purple_loop.py</code> (orchestrator), <code>02_AGENT_PROMPTS.md</code> (all six system prompts), <code>03_STATE_SCHEMA.json</code> , <code>04_SCORING_RUBRIC.md</code> , <code>myskillos_package/</code>
Execution environment	Any OS with Python 3.10+; no API key, network access or GPU required for the default run

Brief installation and usage steps for running the project

1. Extract the submission archive and open the `implementation/` folder.
2. `python purple_loop.py -self-test` — asserts the ten architectural invariants. Expected: 10/10 checks passed.
3. `python purple_loop.py -target sample_target.md` — runs a full engagement and writes `sample_run_report.md` and `final_state.json`. No API key needed.
4. Optional: `export ANTHROPIC_API_KEY=...` then add `-live` to drive the same orchestration with real LLM agents.
5. Optional: `python purple_loop.py -build-package` regenerates the Myskillos-importable agent team from the prompt document.

A significant technical problem encountered and the solution applied

Problem — infinite adversarial escalation. The CHALLENGE verdict is the mechanism that makes the system a loop rather than a pipeline: the defender's patch becomes the attacker's next target. In the first working version this never terminated. Each successful bypass was recorded as a *new* finding, which the defender then mitigated, which the auditor then challenged, which the attacker then bypassed. The engagement consumed its entire budget on a single ancestral finding and the score oscillated instead of converging.

Solution — make “no bypass found” a first-class, scored outcome. Two changes. First, a reworked (version 2) layered control resists the bypass attempt, and the attacker is required to report that failure honestly through its `self_rejected` channel rather than manufacturing a weaker finding to appear productive. Second, the auditor re-audits the challenged pair once the challenge has been answered: if a bypass was produced the verdict is `INSUFFICIENT`, and if none was produced the verdict is `SOUND` with a **durability bonus** of +3. The bonus is the only way in the rubric to earn points for a defence being *tested* rather than merely asserted, so the termination fix and the scoring model reinforce each other instead of competing. Turn 8 of the bundled run is exactly this case: the attacker produces nothing and the defender scores for it.

6 Tests and Results

Describe the basic checks performed to demonstrate that the project works.

	Trial / test	Result obtained	Status
1	Invariant suite: <code>-self-test</code> asserts all five state invariants, the verdict vocabulary, guardian redaction, prompt loading and state projection	10/10 checks passed; attempts to bypass the guardian or to let the auditor author findings raise InvariantError	Pass
2	Full engagement: 12-round run against <code>sample_target.md</code> in mock mode	Ran to completion; 3 findings, 6 mitigations, 8 verdicts, 0 rejected patches; final score 82.6/100 (B)	Pass
3	Non-pipeline assertion: executed agent sequence compared against the fixed cycle Recon/Red/Blue/Audit	Sequence diverges from the fixed cycle; Red is called three times with three different mandates	Pass
4	Safety Guardian: a candidate finding containing operational text is submitted to the filter	Action REDACT ; the operational phrasing is replaced by the vulnerability class and the ATT&CK reference	Pass
5	Myskillos platform dry run: the six-agent team was built on the visual editor, saved and tested	<i>“No issues found — the structure is consistent”</i> ; export reports full compatibility, 8 files	Pass

Overall result: To what extent did the project achieve its objective?

The objective — an orchestration whose control flow is a reasoning product rather than a script — was achieved and is machine-verifiable rather than merely claimed. Three moments in the bundled run are unreachable in a static pipeline: round 5, which exists only because a verdict sent control *backwards* from judge to attacker; round 8, where the attacker correctly produces nothing and the defender earns a durability bonus for surviving; and rounds 9–11, where a defender that regressed was caught twice by the same rubric. All three technical expectations of the course are implemented and covered by the self-test suite. The scoring model, the six-verdict vocabulary and the calibration ledger together make each score traceable to a specific recorded judgment.

Known limitations or incomplete aspects

- **Routing bias (measured, not hidden).** The policy prioritises answering outstanding verdicts over exploring untouched coverage, so a run with many challenge loops can exhaust its budget having probed only one lane. The bundled run ends at 0.30 attack-surface coverage. The fix is to reserve a fraction of the budget for exploration; it was left visible because a routing policy with a demonstrated failure mode is more informative than one never measured.
- **Mock brain.** The offline stand-in exercises the control flow faithfully, but its security *content* is fixed text; only `-live` produces genuine agent reasoning.
- **Not empirical.** LLM agents are not penetration testers. Scores are relative evaluations of an architecture *description*, never measurements of a running system.
- **Heuristic thresholds.** The 0.25 novelty floor and 3.0 score-delta epsilon were chosen by judgment and would need tuning against real engagements.
- Adversarial self-critique reduces but does not eliminate agreement bias; the novelty floor and lane switching are mitigations, not proofs.

7 Security and Responsible Use

Evaluate only the items relevant to your project.

OK Check

- ✓ Secret keys and passwords were not included in the source code.
- ✓ User data was processed only to the extent necessary.
- ✓ Risky or irreversible operations were restricted.
- ✓ Sources and third-party content used were identified.

Project-specific security, ethics, privacy, or copyright concerns and the measures taken

The central concern in a Red-versus-Blue simulation is that an attacker agent could be induced to emit operationally useful attack content. The measure taken is architectural rather than advisory: a dedicated **safety-guardian** agent inspects every attacker output before it can enter shared state, and the patch validator rejects any attacker content that has not been cleared, so the filter cannot be skipped by a malformed response. Attack reasoning is therefore constrained to vulnerability *classes*, MITRE ATT&CK technique identifiers and kill-chain narratives; exploit code, payloads, command lines, malware, evasion tradecraft and ready-to-send phishing text are out of scope by construction. The guardian is instructed to prefer redaction over rejection so analytical value is preserved, but the code and malware rules are reject-only.

The evaluation target is fictional (“NovaPay”) and the guardian additionally replaces any real, named organisation, product version or person with a generic placeholder, so the system cannot be pointed at a real organisation through the input alone. This constraint deliberately reduces attack specificity: the project accepts a small loss of analytical fidelity for a safety property treated as non-negotiable.

No personal data is processed and no credentials appear in the source. The API key, when `-live` is used, is read from the environment and never written to state, logs or the report. Third-party content is limited to the conceptual ATT&CK and STRIDE vocabularies, which are cited in the references and not redistributed.

8 Conclusion and Future Work

The most important results and learning outcomes obtained from the project

The main result is that adversarial multi-agent orchestration can be made *auditable*. Because every routing decision carries a rationale and a falsifiable prediction, and every verdict carries a rationale that cites state, the final score is defensible line by line rather than being an opaque number.

The main learning outcome was that the difficult part of multi-agent design is not prompting the agents — it is deciding what the deterministic layer must own. Placing the legal-move computation, the state invariants and the scoring arithmetic in code, and leaving only judgment to the model, is what made the system both non-trivial and debuggable. A second lesson came from the termination bug: self-critiquing agent teams fail in two opposite directions — agreeing too early, and escalating forever — and both needed an explicit countermeasure, the novelty floor for the first and the durability bonus for the second.

Improvements that could be made with more time

- **Budgeted exploration.** Reserve a fraction of the round budget for coverage so challenge loops cannot starve an entire attack lane — the one measured weakness of the current routing policy.
- **Parallel Red lanes.** Run the logic and social lanes concurrently and let the Router arbitrate between their findings, rather than switching sequentially.
- **Calibration over many runs.** The confidence multipliers currently reset each engagement; persisting them across runs would let agent reliability be learned rather than assumed.
- **Threshold tuning.** Fit the novelty floor and convergence epsilon against a corpus of real architecture reviews instead of choosing them by judgment.
- **Human-in-the-loop review.** Expose the HUMAN escalation as an interactive prompt so an operator can supply missing architecture detail mid-run.

Core course concepts applied in the project

State management — a single JSON blackboard with patch-based writes, an append-only history, per-agent read projections, context compaction and five enforced invariants. **Reflection and feedback loops** — three nested loops: intra-agent draft/critique/revise, a six-verdict adversarial cross-critique in which the CHALLENGE verdict re-routes control backwards, and engagement-level convergence with cross-round confidence calibration. **Dynamic routing** — an LLM router operating over a formally constrained legal-move set, generating mandates rather than mere destinations, and making falsifiable predictions that are graded the following round. Alongside these: the orchestrator-worker pattern, separation of powers between generator and judge, guardrails as an architectural component, and structured JSON contracts between agents.

9 References and Submission Checklist

Documents, articles, datasets, models, open-source projects, or other resources used

- MITRE ATT&CK Enterprise technique taxonomy — <https://attack.mitre.org> (used as a vocabulary for referring to techniques abstractly; not redistributed).
- STRIDE threat categorisation (Microsoft) — used as the structure of the `coverage_map`.
- Anthropic Claude documentation and Messages API — <https://docs.claude.com> (model `claude-sonnet-5`, used in `-live` mode).
- Myskillos agent platform — <https://myskillos.com> (skill packaging format: `CLAUDE.md` plus `.claude/agents/*.md`).
- SENG 456 course materials on agent orchestration, state management and dynamic routing.
- AI assistance: Claude was used to help design and implement the orchestration, as explicitly permitted by the project brief. All architectural decisions, the verdict vocabulary, the scoring model and the final content of this report were reviewed and are owned by the author.

OK Submission item

- ✓ A completed PDF version of this form is included.
 - ✓ The source code or project files are accessible.
 - ✓ Installation and execution instructions are included.
 - ✓ A test result or sample usage result is included.
 - ✓ No secret keys, passwords, or unauthorized personal data are included.
 - ✓ Demo, video, or supplementary file links work, if provided.
-

10 Student Declaration

I declare that I prepared this project individually; identified the sources, third-party code, and artificial intelligence assistance I used; and ensured that the submission files contain no secret keys or unauthorized personal data.

Full Name Mohammed Mustafa Kareem

Student ID 230208878

Date 19 August 2026

Signature _____